HARDWARE DESIGN DOCUMENT

APB Packet Switch with Packet Generator

Design specification and SystemVerilog RTL listing
AMBA APB3 · 4 ports · 25-register CSR map

IP name	`apb_pkt_switch`
Bus protocol	AMBA 3 APB (32-bit data, 16-bit address)
Datapath	Packet generator + 4-port dest-lookup switch
Registers	25 (RO / WO / RW / W1C) — status, error, configuration
Language	IEEE 1800 SystemVerilog
UVM RAL	`pkt_reg_block` in `ral/pkt_ral_pkg.sv`

Unlock key: `32'hA5A5_C3C3`. The CSR bank powers up locked.

APB Packet Switch with Packet Generator

1. Overview

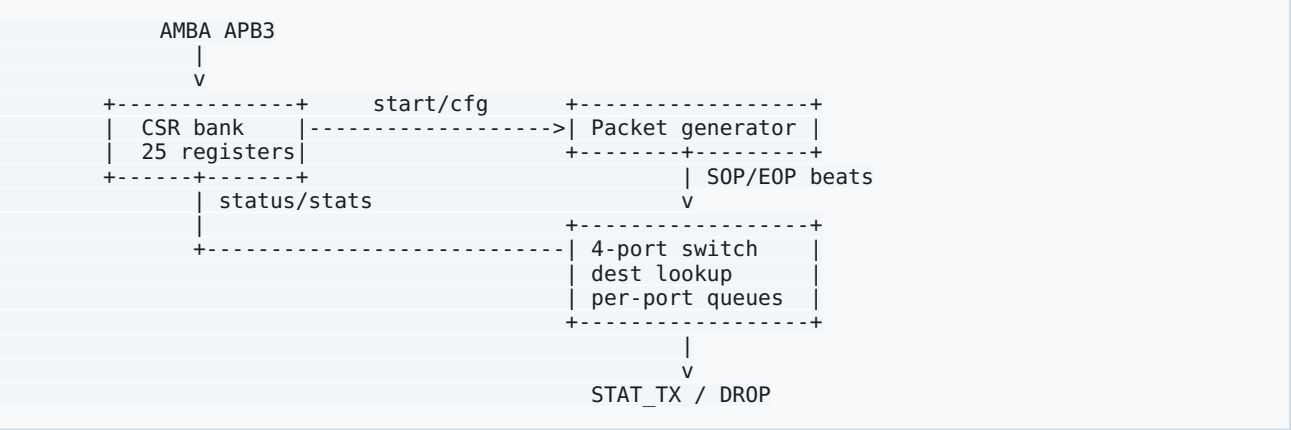
`apb_pkt_switch` is a **4-port packet switch** with an on-chip **packet generator**. Software programs 25 CSRs through an **AMBA 3 APB** slave. The generator injects packets into the fabric; the switch forwards them to a destination port, counts RX/TX/drops, and reports errors.

Property	Value
Bus	AMBA 3 APB, 32-bit data, 16-bit byte address
Ports	4 (`0..3`)
Queue depth per dest port	16 beats
Packet beat	32-bit word, SOP/EOP framed
Registers	25 (RO / WO / RW / W1C)
Address span	`0x00` – `0x60`
Wait states	0 (`pready=1`)
Reset	`presetn` async active-low
Unlock key	`32'hA5A5C3C3` (powers up locked)

Typical program sequence: unlock → enable ports / forwarding → `SWITCH_CTRL.ENABLE` → program `GEN_PKT_CFG` / `GEN_COUNT` → `GEN_CTRL.ENABLE+START` → poll `GEN_STATUS` and `STAT_TX`.

---

2. Block diagram



The generator is the only traffic source in this slice (other ingress ports are unused). Destination is taken from `GEN_PKT_CFG.DST_PORT`.

---

3. Register width, addressing, and APB operation

3.1 Register width and byte addressing

Every register in this design is 32 bits wide. The APB data bus is therefore also 32 bits wide. `paddr` is a **byte address**, and registers are placed every 4 bytes:

```
Register N address = base address + (N × 4)
```

Consequently, all valid register addresses have `paddr[1:0] = 2'b00`. For example, register 0 is at `0x00`, register 1 at `0x04`, and register 24 at `0x60`.

This interface is APB3 and does not implement APB4 `PSTRB`. Each write is a complete 32-bit transfer. Field write masks inside the RTL ensure that unimplemented/reserved bits are ignored.

3.2 APB interface

Signal	Dir	Width	Description
`pclk`	in	1	APB clock. Register writes occur on its rising edge in the access phase.
`presetn`	in	1	Asynchronous active-low reset.
`psel`	in	1	Selects this APB slave.
`penable`	in	1	Distinguishes setup (`0`) from access (`1`) phase.
`pwrite`	in	1	`1` for write; `0` for read.
`paddr`	in	16	Byte address. Must be 32-bit aligned.
`pwdata`	in	32	Write data, valid throughout setup and access phases.
`prdata`	out	32	Read data, combinational from the selected register.
`pready`	out	1	Always `1` ; every transfer completes with zero wait states.
`pslverr`	out	1	High in the access phase for an unaligned or out-of-range address.
`irq`	out	1	Level-sensitive interrupt output.

3.3 APB write transfer

An APB write uses two clock cycles:

Phase	PSEL	PENABLE	PWRITE	Master action	Slave action
Setup	1	0	1	Drives stable `PADDR` and `PWDATA` .	Decodes address; does not update state.
Access	1	1	1	Holds address/data stable.	With `PREADY=1` , captures the write on the rising edge.

The internal write qualifier is:

```
wr_ok = psel && penable && pready && pwrite &&
        (paddr[1:0] == 2'b00) && (paddr <= 16'h0060);
```

Example—unlock the register bank:

Cycle	PSEL	PENABLE	PWRITE	PADDR	PWDATA	Result
T0 setup	1	0	1	0x005C	0xA5A5C3C3	No state change yet
T1 access	1	1	1	0x005C	0xA5A5C3C3	LOCKED becomes 0

3.4 APB read transfer

Phase	PSEL	PENABLE	PWRITE	Master action	Slave action
Setup	1	0	0	Drives stable PADDR.	Combinationally selects PRDATA.
Access	1	1	0	Samples PRDATA when PREADY=1.	Holds selected value on PRDATA.

Example—read transmitted packet count:

Cycle	PSEL	PENABLE	PWRITE	PADDR	PRDATA
T0 setup	1	0	0	0x0018	Current STAT_TX value
T1 access	1	1	0	0x0018	Master samples the value

3.5 Error and illegal-access behavior

Access	PSLVERR	Result
Aligned address 0x00 – 0x60	0	Normal access according to field policy.
Unaligned address such as 0x02	1	Write has no effect; read returns 0.
Address greater than 0x60	1	Write has no effect; read returns 0.
Write to RO register	0	Legal transfer; write is ignored.
Read from WO register	0	Legal transfer; returns 0.
Write lockable register while locked	0	Legal transfer; write is ignored.

PSLVERR is meaningful only during the APB access phase (PSEL && PENABLE).

3.6 Reset, reserved bits, and access policies

Policy	Definition
RO	Software cannot modify it. Hardware supplies the value.
RW	Software reads and writes it. A lockable RW field accepts writes only when LOCKED=0.
WO	A write produces an event/pulse; reads return zero.
W1C	Writing a 1 clears the sticky bit; writing 0 leaves it unchanged.
RAZ/WI	Reserved field reads as zero and ignores writes.

All reserved fields are **RAZ/WI**. Hard reset (presetn=0) asynchronously resets the CSR storage and datapath. SOFT_RESET.PULSE synchronously restores the same programmable-state defaults.

The block powers up locked. Lockable registers are SWITCH_CTRL, PORT_ENABLE, FWD_ENABLE, GEN_CTRL, GEN_PKT_CFG, GEN_INTERVAL, GEN_COUNT, IRQ_ENABLE, and ERR_ENABLE. W1C registers and diagnostic WO registers remain accessible while locked.

For IRQ_STATUS and ERR_STATUS, this RTL calculates (old | hardware_set) & ~software_clear; therefore a W1C clear has priority if a hardware set and software clear occur in the same clock cycle.

4. Complete register map

All entries below have a **register width of 32 bits** and a **4-byte address size**.

No.	Offset	Name	Register access	Width	Reset	Lockable	Function
1	0x00	DEVICE_ID	RO	32	0x02A55357	No	Product identification
2	0x04	VERSION	RO	32	0x00010200	No	RTL version 1.2.0
3	0x08	SWITCH_STATUS	RO	32	0x000000F9	No	Overall live status
4	0x0C	PORT_STATUS	RO	32	0x00000000	No	Four queue levels
5	0x10	GEN_STATUS	RO	32	0x00000000	No	Generator state
6	0x14	STAT_RX	RO	32	0x00000000	No	Accepted packet count
7	0x18	STAT_TX	RO	32	0x00000000	No	Forwarded packet count
8	0x1C	STAT_DROP	RO	32	0x00000000	No	Dropped packet count
9	0x20	SWITCH_CTRL	RW/WO	32	0x00000000	Yes	Fabric and IRQ control
10	0x24	PORT_ENABLE	RW	32	0x0000000F	Yes	Per-port enable
11	0x28	FWD_ENABLE	RW	32	0x0000000F	Yes	Destination forwarding mask
12	0x2C	GEN_CTRL	RW/WO	32	0x00000000	Yes	Generator run control
13	0x30	GEN_PKT_CFG	RW	32	0x00000401	Yes	Packet format
14	0x34	GEN_INTERVAL	RW	32	0x00000002	Yes	Inter-packet gap
15	0x38	GEN_COUNT	RW	32	0x00000004	Yes	Packets per run
16	0x3C	IRQ_STATUS	W1C	32	0x00000000	No	Sticky interrupt events
17	0x40	IRQ_ENABLE	RW	32	0x00000000	Yes	Interrupt mask
18	0x44	IRQ_FORCE	WO	32	0x00000000	No	Software interrupt set
19	0x48	ERR_STATUS	W1C	32	0x00000000	No	Sticky errors
20	0x4C	ERR_ENABLE	RW	32	0x00000000	Yes	Error-to-IRQ mask
21	0x50	ERR_INJECT	WO	32	0x00000000	No	Error diagnostics
22	0x54	ERR_COUNT	RO	32	0x00000000	No	Saturating error counter
23	0x58	ERR_INFO	RO	32	0x00000000	No	Last-error snapshot
24	0x5C	LOCK	Key write/RO	32	0x00000001	No	Configuration lock
25	0x60	SOFT_RESET	WO	32	0x00000000	No	Synchronous internal reset

5. Detailed field descriptions

5.1 DEVICE_ID — offset `0x00`, 32-bit, RO, reset `0x02A55357`

Bits	Width	Field	Access	Reset	Description
31:24	8	`REVISION`	RO	`0x02`	Hardware/RTL product revision.
23:16	8	`VENDOR_ID`	RO	`0xA5`	Example vendor identifier.
15:0	16	`PRODUCT_ID`	RO	`0x5357`	Product code (`"SW"` in ASCII).

This register is constant. Any APB write completes without error and is ignored.

5.2 VERSION — offset `0x04`, 32-bit, RO, reset `0x00010200`

Bits	Width	Field	Access	Reset	Description
31:24	8	`RESERVED`	RAZ/WI	0	Reserved.
23:16	8	`MAJOR`	RO	1	Major version.
15:8	8	`MINOR`	RO	2	Minor version.
7:0	8	`PATCH`	RO	0	Patch version.

The encoded semantic version is **1.2.0**.

5.3 SWITCH_STATUS — offset `0x08`, 32-bit, RO, reset `0x000000F9`

Bits	Width	Field	Access	Reset	Description
31:18	14	`RESERVED`	RAZ/WI	0	Reserved.
17:16	2	`GEN_FSM`	RO, volatile	0	Generator state: 0 IDLE, 1 SEND, 2 GAP, 3 DONE.
15:10	6	`RESERVED`	RAZ/WI	0	Reserved.
9	1	`ERR_PENDING`	RO, volatile	0	OR-reduction of `ERR_STATUS[7:0]`.
8	1	`GEN_RUNNING`	RO, volatile	0	1 while generator is in SEND or GAP.
7:4	4	`PORT_ENABLED`	RO, volatile	`0xF`	Live mirror of `PORT_ENABLE[3:0]`.
3	1	`LOCKED`	RO, volatile	1	Live configuration-lock state.
2	1	`SWITCH_ENABLED`	RO, volatile	0	Mirror of `SWITCH_CTRL.ENABLE`.
1	1	`BUSY`	RO, volatile	0	Generator running or at least one destination queue non-empty.
0	1	`READY`	RO, volatile	1	Generator is not running; it may be IDLE or DONE.

5.4 PORT_STATUS — offset `0x0C`, 32-bit, RO, reset `0x00000000`

Bits	Width	Field	Access	Reset	Description
31:24	8	`PORT3_LEVEL`	RO, volatile	0	Destination-port 3 queue occupancy in 32-bit beats. Valid range 0–16.
23:16	8	`PORT2_LEVEL`	RO, volatile	0	Destination-port 2 queue occupancy in beats.
15:8	8	`PORT1_LEVEL`	RO, volatile	0	Destination-port 1 queue occupancy in beats.
7:0	8	`PORT0_LEVEL`	RO, volatile	0	Destination-port 0 queue occupancy in beats.

5.5 GEN_STATUS — offset `0x10`, 32-bit, RO, reset `0x00000000`

Bits	Width	Field	Access	Reset	Description
31:24	8	`RESERVED`	RAZ/WI	0	Reserved.
23:8	16	`REMAINING`	RO, volatile	0	Packets remaining in a counted run. In continuous mode it does not decrement.
7:4	4	`RESERVED`	RAZ/WI	0	Reserved.
3:2	2	`FSM`	RO, volatile	0	0 IDLE, 1 SEND, 2 GAP, 3 DONE.
1	1	`DONE`	RO, volatile	0	1 while FSM is in DONE; cleared by a new accepted START, STOP, disabling generator, or reset.
0	1	`RUNNING`	RO, volatile	0	1 in SEND or GAP.

5.6 STAT_RX — offset `0x14`, 32-bit, RO, reset `0x00000000`

Bits	Width	Field	Access	Reset	Description
31:16	16	`RESERVED`	RAZ/WI	0	Reserved.
15:0	16	`RX_PACKETS`	RO, volatile	0	Saturating count of accepted packet EOPs. Maximum `0xFFFF`.

Cleared by hard reset, soft reset, or `SWITCH_CTRL.STAT_CLEAR=1`.

5.7 STAT_TX — offset `0x18`, 32-bit, RO, reset `0x00000000`

Bits	Width	Field	Access	Reset	Description
31:16	16	`RESERVED`	RAZ/WI	0	Reserved.
15:0	16	`TX_PACKETS`	RO, volatile	0	Saturating count of forwarded packet EOPs. Maximum `0xFFFF`.

In this switch slice, a packet accepted from the generator is immediately considered forwarded, so `STAT_RX` and `STAT_TX` increment together.

5.8 STAT_DROP — offset `0x1C`, 32-bit, RO, reset `0x00000000`

Bits	Width	Field	Access	Reset	Description
31:16	16	RESERVED	RAZ/WI	0	Reserved.
15:0	16	DROPPED_PACKETS	RO, volatile	0	Saturating count of packet EOPs dropped because switch/port/forwarding is disabled or the queue is full.

5.9 SWITCH_CTRL — offset `0x20`, 32-bit, mixed RW/WO, reset `0x00000000`

This register is lockable.

Bits	Width	Field	Access	Reset	Description
31:4	28	RESERVED	RAZ/WI	0	Reserved.
3	1	IRQ_GLOBAL_EN	RW	0	Global interrupt gate. If 0, `irq` remains low regardless of status/masks.
2	1	RESERVED	RAZ/WI	0	Reserved.
1	1	STAT_CLEAR	WO pulse	0	Writing 1 clears RX/TX/drop counters, queue levels, and the internal dropping state. Always reads 0.
0	1	ENABLE	RW	0	Enables packet forwarding. When 0, generated packets are dropped and set destination/drop events.

A write such as `0x00000009` enables both switching and global interrupts. To clear statistics while preserving these controls, write `0x0000000B`.

5.10 PORT_ENABLE — offset `0x24`, 32-bit, RW, reset `0x0000000F`

This register is lockable.

Bits	Width	Field	Access	Reset	Description
31:4	28	RESERVED	RAZ/WI	0	Reserved.
3	1	PORT3_EN	RW	1	Enables destination port 3.
2	1	PORT2_EN	RW	1	Enables destination port 2.
1	1	PORT1_EN	RW	1	Enables destination port 1.
0	1	PORT0_EN	RW	1	Enables destination port 0.

A packet targeting a disabled port is dropped. `ERR_STATUS.DEST_ERROR` is set at SOP, and `IRQ_STATUS.DROP` is set at EOP.

5.11 FWD_ENABLE — offset `0x28`, 32-bit, RW, reset `0x0000000F`

This register is lockable.

Bits	Width	Field	Access	Reset	Description
31:4	28	RESERVED	RAZ/WI	0	Reserved.
3	1	FWD3_EN	RW	1	Allows forwarding to destination port 3.
2	1	FWD2_EN	RW	1	Allows forwarding to destination port 2.
1	1	FWD1_EN	RW	1	Allows forwarding to destination port 1.
0	1	FWD0_EN	RW	1	Allows forwarding to destination port 0.

Unlike PORT_ENABLE, this is a forwarding-policy mask. A port can be enabled but administratively blocked from receiving generated traffic.

5.12 GEN_CTRL — offset 0x2C, 32-bit, mixed RW/WO, reset 0x00000000

This register is lockable.

Bits	Width	Field	Access	Reset	Description
31:4	28	RESERVED	RAZ/WI	0	Reserved.
3	1	CONTINUOUS	RW	0	1 ignores the programmed count and generates until STOP or ENABLE=0.
2	1	STOP	WO pulse	0	Writing 1 immediately returns the generator FSM to IDLE and clears remaining count. Reads 0.
1	1	START	WO pulse	0	Writing 1 requests a run. Accepted only in IDLE or DONE while effective ENABLE=1. Reads 0.
0	1	ENABLE	RW	0	Enables the generator. Clearing it forces IDLE.

ENABLE and START may be asserted in one APB write (0x00000003). START_ACK is then set. The stored readback is 0x00000001 because START is a pulse.

5.13 GEN_PKT_CFG — offset 0x30, 32-bit, RW, reset 0x00000401

This register is lockable.

Bits	Width	Field	Access	Reset	Description
31:24	8	RESERVED	RAZ/WI	0	Reserved.
23:16	8	SEED	RW	0	Upper payload byte.
15:8	8	LENGTH	RW	4	Requested packet length in 32-bit beats. 0 is stored as 1; values above 16 remain readable but the generator clamps the effective length to 16.
7:4	4	RESERVED	RAZ/WI	0	Reserved.
3:2	2	SRC_PORT	RW	0	Source-port tag placed on the internal packet sideband.
1:0	2	DST_PORT	RW	1	Destination port selected by the switch. Values 0–3 are valid.

Each generated data beat is:

```
{SEED[7:0], packet_index[7:0], beat_index[7:0], 8'hA5}
```

Configuration should not be changed while GEN_STATUS.RUNNING=1; the generator uses live register values.

5.14 GEN_INTERVAL — offset `0x34`, 32-bit, RW, reset `0x00000002`

This register is lockable.

Bits	Width	Field	Access	Reset	Description
31:16	16	`RESERVED`	RAZ/WI	0	Reserved.
15:0	16	`INTERVAL`	RW	2	Number of GAP-state clock cycles between packet EOP and the next packet SOP. 0 starts the next packet without entering GAP.

5.15 GEN_COUNT — offset `0x38`, 32-bit, RW, reset `0x00000004`

This register is lockable.

Bits	Width	Field	Access	Reset	Description
31:16	16	`RESERVED`	RAZ/WI	0	Reserved.
15:0	16	`PACKET_COUNT`	RW	4	Packets generated by one accepted START in counted mode. Programmed 0 is interpreted as 1 at START. Ignored in continuous mode.

5.16 IRQ_STATUS — offset `0x3C`, 32-bit, W1C, reset `0x00000000`

This register is accessible while locked.

Bits	Width	Field	Access	Reset	Set condition
31:8	24	`RESERVED`	RAZ/WI	0	Reserved.
7	1	`SW_EVENT`	W1C	0	No hardware source; may be set through `IRQ_FORCE[7]`.
6	1	`START_ACK`	W1C	0	Generator accepts START.
5	1	`QUEUE_ALMOST_FULL`	W1C	0	Any queue level is at least 8. This condition is level-derived and reasserts while true.
4	1	`PACKET_FORWARDED`	W1C	0	Accepted packet reaches EOP.
3	1	`PACKET_DROPPED`	W1C	0	Dropped packet reaches EOP.
2	1	`QUEUE_OVERFLOW`	W1C	0	Destination queue is full when packet SOP arrives.
1	1	`ERROR`	W1C	Any enabled sticky error exists: `|(ERR_STATUS & ERR_ENABLE)`. Reasserts until the enabled error is cleared/masked.	
0	1	`GEN_DONE`	W1C	Last packet of a non-continuous counted run reaches EOP.	

Example: to clear DONE and START_ACK without affecting other bits, APB-write `0x00000041` to offset `0x3C`.

5.17 IRQ_ENABLE — offset `0x40`, 32-bit, RW, reset `0x00000000`

This register is lockable. Fields correspond one-for-one with `IRQ_STATUS`.

Bits	Width	Field	Access	Reset	Description
31:8	24	`RESERVED`	RAZ/WI	0	Reserved.
7	1	`SW_EVENT_EN`	RW	0	Enables forced software event interrupt.
6	1	`START_ACK_EN`	RW	0	Enables START-ack interrupt.
5	1	`QUEUE_AF_EN`	RW	0	Enables queue-almost-full interrupt.
4	1	`PACKET_FWD_EN`	RW	0	Enables forwarded-packet interrupt.
3	1	`PACKET_DROP_EN`	RW	0	Enables dropped-packet interrupt.
2	1	`OVERFLOW_EN`	RW	0	Enables overflow interrupt.
1	1	`ERROR_EN`	RW	0	Enables aggregated error interrupt.
0	1	`GEN_DONE_EN`	RW	0	Enables generator-done interrupt.

The output equation is:

```
irq = SWITCH_CTRL.IRQ_GLOBAL_EN &&
      |(IRQ_STATUS[7:0] & IRQ_ENABLE[7:0]);
```

5.18 IRQ_FORCE — offset `0x44`, 32-bit, WO, reset/read value `0x00000000`

This register is accessible while locked.

Bits	Width	Field	Access	Reset	Description
31:8	24	`RESERVED`	WI	0	Ignored.
7:0	8	`FORCE`	WO	0	Each written 1 sets the corresponding `IRQ_STATUS` bit. Written 0 has no effect.

The force register does not bypass `IRQ_ENABLE` or `IRQ_GLOBAL_EN`; it only sets status.

5.19 ERR_STATUS — offset `0x48`, 32-bit, W1C, reset `0x00000000`

This register is accessible while locked.

Bits	Width	Field	Access	Reset	Set condition
31:8	24	`RESERVED`	RAZ/WI	0	Reserved.
7	1	`INJECTED`	W1C	0	Set by any `ERR_INJECT[7:0]` event.
6	1	`LENGTH_ERROR`	W1C	0	Diagnostic injection only in this RTL.
5	1	`CRC_ERROR`	W1C	0	Diagnostic injection only.
4	1	`PROTOCOL_ERROR`	W1C	0	Diagnostic injection only.
3	1	`UNDERSIZE`	W1C	0	Diagnostic injection only.
2	1	`QUEUE_OVERFLOW`	W1C	0	Destination queue full at packet SOP or injection.
1	1	`DEST_ERROR`	W1C	0	Switch disabled, destination port disabled, or destination forwarding disabled at SOP; or injection.
0	1	`BUS_ERROR`	W1C	0	Diagnostic injection only.

5.20 ERR_ENABLE — offset `0x4C`, 32-bit, RW, reset `0x00000000`

This register is lockable. Each bit enables promotion of the corresponding `ERR_STATUS` bit to `IRQ_STATUS.ERROR`.

Bits	Width	Field	Access	Reset	Description
31:8	24	`RESERVED`	RAZ/WI	0	Reserved.
7	1	`INJECTED_EN`	RW	0	Enable injected-summary promotion.
6	1	`LENGTH_EN`	RW	0	Enable length-error promotion.
5	1	`CRC_EN`	RW	0	Enable CRC-error promotion.
4	1	`PROTOCOL_EN`	RW	0	Enable protocol-error promotion.
3	1	`UNDERSIZE_EN`	RW	0	Enable undersize promotion.
2	1	`OVERFLOW_EN`	RW	0	Enable queue-overflow promotion.
1	1	`DEST_EN`	RW	0	Enable destination-error promotion.
0	1	`BUS_EN`	RW	0	Enable bus-error promotion.

5.21 ERR_INJECT — offset `0x50`, 32-bit, WO, reset/read value `0x00000000`

This register is accessible while locked.

Bits	Width	Field	Access	Reset	Description
31	1	`CLEAR_COUNT`	WO	0	Writing 1 clears `ERR_COUNT` and `ERR_INFO`. This clear has priority over an injection in the same write.
30:8	23	`RESERVED`	WI	0	Ignored.
7	1	`INJECTED`	WO	0	Sets `ERR_STATUS.INJECTED`.
6	1	`LENGTH`	WO	0	Sets LENGTH_ERROR and INJECTED.
5	1	`CRC`	WO	0	Sets CRC_ERROR and INJECTED.
4	1	`PROTOCOL`	WO	0	Sets PROTOCOL_ERROR and INJECTED.
3	1	`UNDERSIZE`	WO	0	Sets UNDERSIZE and INJECTED.
2	1	`OVERFLOW`	WO	0	Sets QUEUE_OVERFLOW and INJECTED.
1	1	`DEST`	WO	0	Sets DEST_ERROR and INJECTED.
0	1	`BUS`	WO	0	Sets BUS_ERROR and INJECTED.

For injected bits 0–6, `ERR_COUNT` increases by the population count (number of 1s). A bit-7-only injection increments the counter by one. The counter saturates at `0xFFFF`.

5.22 ERR_COUNT — offset `0x54`, 32-bit, RO, reset `0x00000000`

Bits	Width	Field	Access	Reset	Description
31:16	16	`RESERVED`	RAZ/WI	0	Reserved.
15:0	16	`ERROR_COUNT`	RO, volatile	0	Saturating count of hardware error events and injected error bits.

5.23 ERR_INFO — offset `0x58`, 32-bit, RO, reset `0x00000000`

Bits	Width	Field	Access	Reset	Description
31:20	12	TX_SNAPSHOT	RO, volatile	0	Low 12 bits of STAT_TX at the most recent error capture.
19:18	2	RESERVED	RAZ/WI	0	Reserved.
17:16	2	DEST_PORT	RO, volatile	0	Generator destination at capture time.
15:8	8	SOURCE	RO, volatile	0 for hardware event; 1 for injection.	
7:0	8	LAST_CODE	RO, volatile	Index of lowest set captured error bit (0–7).	

If several error bits arrive together, LAST_CODE records the lowest-numbered bit.

5.24 LOCK — offset `0x5C`, 32-bit, key-write/read-status, reset `0x00000001`

Bits	Width	Field	Access	Reset	Description
31:1	31	KEY/ RESERVED	WO on write, read 0	0	Participates in the 32-bit key comparison; reads zero.
0	1	LOCKED	RO on read	1	1 blocks writes to lockable registers; 0 allows them.

Write behavior is based on the complete 32-bit value:

APB write data	Result
0xA5A5C3C3	Unlock (LOCKED=0)
Any other value	Lock (LOCKED=1)

The key is never returned by a read.

5.25 SOFT_RESET — offset `0x60`, 32-bit, WO, reset/read value `0x00000000`

Bits	Width	Field	Access	Reset	Description
31:1	31	RESERVED	WI	0	Ignored.
0	1	PULSE	WO	0	Writing 1 synchronously resets CSR state, locks configuration, stops generator, and clears queues/statistics.

Writing zero has no effect. Reading always returns zero. This command remains available while locked.

---

6. Programming examples

6.1 Generate four packets from source 0 to destination 1

Every operation below is a 32-bit APB write.

Step	Address	Data	Purpose
1	`0x5C`	`0xA5A5C3C3`	Unlock configuration.
2	`0x24`	`0x0000000F`	Enable all four ports.
3	`0x28`	`0x0000000F`	Allow all destinations.
4	`0x20`	`0x00000009`	Enable switch and global interrupt.
5	`0x30`	`0x00100401`	Seed `0x10`, length 4, source 0, destination 1.
6	`0x34`	`0x00000002`	Two-cycle inter-packet gap.
7	`0x38`	`0x00000004`	Generate four packets.
8	`0x40`	`0x00000041`	Enable GEN_DONE and START_ACK interrupts.
9	`0x2C`	`0x00000003`	Set ENABLE and pulse START.

After completion, expected values include `STAT_RX=4`, `STAT_TX=4`, `STAT_DROP=0`, `GEN_STATUS.DONE=1`, and `IRQ_STATUS.GEN_DONE=1`.

6.2 Clear sticky status

To clear `IRQ_STATUS.GEN_DONE` (bit 0) and `START_ACK` (bit 6), write `0x00000041` to `0x3C`. To clear all implemented IRQ bits, write `0x000000FF`.

To clear `ERR_STATUS.DEST_ERROR` only, write `0x00000002` to `0x48`.

6.3 Cause and observe a destination drop

1. Unlock and enable the switch.
2. Write `FWD_ENABLE=0x00000001` so only destination 0 is allowed.
3. Program `GEN_PKT_CFG.DST_PORT=1`.
4. Start the generator.

The packet is dropped. At SOP, `ERR_STATUS.DEST_ERROR` sets; at EOP, `STAT_DROP` increments and `IRQ_STATUS.PACKET_DROPPED` sets.

---

7. Datapath behaviour

Generator FSM: IDLE → SEND (LENGTH beats) → GAP (INTERVAL) → SEND … → DONE.

Switch: On each valid beat, destination = `DST_PORT`. If switch disabled, dest port disabled, or `FWD_ENABLE[dst]=0`, the packet is dropped (dropping holds until EOP). If the dest queue is at 16 beats at SOP, `OVERFLOW` and drop. Accepted beats increment that port’s occupancy; occupancy drains one beat per cycle on ports that are not currently receiving.

A counted run of 4 packets, length 4, dest 1, all enables on, increments `STAT_RX=STAT_TX=4`, `STAT_DROP=0`.

---

8. UVM RAL

Block class `pkt_reg_block` in `ral/pkt_ral_pkg.sv`. Map `apb_map`, 4 bytes, little endian. Adapter: `ral/apb_reg_adapter.sv`. HDL root `tb.dut`.

---

9. Files

Path	Role
`rtl/apb_pkt_pkg.sv`	Offsets and resets
`rtl/pkt_generator.sv`	Packet generator
`rtl/pkt_switch.sv`	4-port switch
`rtl/apb_pkt_switch.sv`	APB CSR top
`ral/pkt_ral_pkg.sv`	UVM RAL
`tb/tb_apb_pkt_switch.sv`	Directed APB tests

Appendix A — SystemVerilog RTL source

Complete synthesizable RTL from `rtl/`.

`rtl/apb_pkt_pkg.sv — Address map, reset values, field masks`

```
// ------------------------------------------------------------------------------
// apb_pkt_pkg.sv
// Constants for the APB 4-port packet switch + packet generator.
// ------------------------------------------------------------------------------
package apb_pkt_pkg;

  localparam int unsigned ADDR_W     = 16;
  localparam int unsigned DATA_W     = 32;
  localparam int unsigned N_PORTS    = 4;
  localparam int unsigned Q_DEPTH    = 16;
  localparam int unsigned LEN_MAX    = 16;

  localparam logic [ADDR_W-1:0] ADDR_DEVICE_ID     = 16'h0000;
  localparam logic [ADDR_W-1:0] ADDR_VERSION       = 16'h0004;
  localparam logic [ADDR_W-1:0] ADDR_SWITCH_STATUS = 16'h0008;
  localparam logic [ADDR_W-1:0] ADDR_PORT_STATUS   = 16'h000C;
  localparam logic [ADDR_W-1:0] ADDR_GEN_STATUS    = 16'h0010;
  localparam logic [ADDR_W-1:0] ADDR_STAT_RX       = 16'h0014;
  localparam logic [ADDR_W-1:0] ADDR_STAT_TX       = 16'h0018;
  localparam logic [ADDR_W-1:0] ADDR_STAT_DROP     = 16'h001C;
  localparam logic [ADDR_W-1:0] ADDR_SWITCH_CTRL   = 16'h0020;
  localparam logic [ADDR_W-1:0] ADDR_PORT_ENABLE   = 16'h0024;
  localparam logic [ADDR_W-1:0] ADDR_FWD_ENABLE    = 16'h0028;
  localparam logic [ADDR_W-1:0] ADDR_GEN_CTRL      = 16'h002C;
  localparam logic [ADDR_W-1:0] ADDR_GEN_PKT_CFG   = 16'h0030;
  localparam logic [ADDR_W-1:0] ADDR_GEN_INTERVAL  = 16'h0034;
  localparam logic [ADDR_W-1:0] ADDR_GEN_COUNT     = 16'h0038;
  localparam logic [ADDR_W-1:0] ADDR_IRQ_STATUS    = 16'h003C;
  localparam logic [ADDR_W-1:0] ADDR_IRQ_ENABLE    = 16'h0040;
  localparam logic [ADDR_W-1:0] ADDR_IRQ_FORCE     = 16'h0044;
  localparam logic [ADDR_W-1:0] ADDR_ERR_STATUS    = 16'h0048;
  localparam logic [ADDR_W-1:0] ADDR_ERR_ENABLE    = 16'h004C;
  localparam logic [ADDR_W-1:0] ADDR_ERR_INJECT    = 16'h0050;
  localparam logic [ADDR_W-1:0] ADDR_ERR_COUNT     = 16'h0054;
  localparam logic [ADDR_W-1:0] ADDR_ERR_INFO      = 16'h0058;
  localparam logic [ADDR_W-1:0] ADDR_LOCK          = 16'h005C;
  localparam logic [ADDR_W-1:0] ADDR_SOFT_RESET    = 16'h0060;
  localparam logic [ADDR_W-1:0] ADDR_MAX           = 16'h0060;

  localparam logic [DATA_W-1:0] RST_DEVICE_ID   = 32'h02A5_5357; // rev 2, vendor A5, "SW"
  localparam logic [DATA_W-1:0] RST_VERSION     = 32'h0001_0200; // 1.2.0
  localparam logic [DATA_W-1:0] RST_PORT_ENABLE = 32'h0000_000F;
  localparam logic [DATA_W-1:0] RST_FWD_ENABLE  = 32'h0000_000F;
  localparam logic [DATA_W-1:0] RST_GEN_PKT_CFG = 32'h0000_0401; // len=4, src=0, dst=1
  localparam logic [DATA_W-1:0] RST_GEN_INTERVAL= 32'h0000_0002;
  localparam logic [DATA_W-1:0] RST_GEN_COUNT   = 32'h0000_0004;

  localparam logic [DATA_W-1:0] UNLOCK_KEY = 32'hA5A5_C3C3;

  localparam logic [DATA_W-1:0] MASK_SWITCH_CTRL  = 32'h0000_0009; // ENABLE + IRQ_GLOBAL_EN; STAT_CLR is pulse
  localparam logic [DATA_W-1:0] MASK_PORT_ENABLE  = 32'h0000_000F;
```

```
  localparam logic [DATA_W-1:0] MASK_FWD_ENABLE   = 32'h0000_000F;
  localparam logic [DATA_W-1:0] MASK_GEN_CTRL     = 32'h0000_0009; // ENABLE + CONTINUOUS; START/STOP pulses
  localparam logic [DATA_W-1:0] MASK_GEN_PKT_CFG  = 32'h00FF_FF0F;
  localparam logic [DATA_W-1:0] MASK_GEN_INTERVAL = 32'h0000_FFFF;
  localparam logic [DATA_W-1:0] MASK_GEN_COUNT    = 32'h0000_FFFF;
  localparam logic [DATA_W-1:0] MASK_IRQ          = 32'h0000_00FF;
  localparam logic [DATA_W-1:0] MASK_ERR          = 32'h0000_00FF;

  localparam logic [1:0] GEN_IDLE = 2'd0;
  localparam logic [1:0] GEN_SEND = 2'd1;
  localparam logic [1:0] GEN_GAP  = 2'd2;
  localparam logic [1:0] GEN_DONE = 2'd3;

endpackage
```

rtl/pkt_generator.sv — Programmable packet generator

```
// ------------------------------------------------------------------------------
// pkt_generator.sv
// Programmable packet generator: emits counted packets onto the switch fabric.
// ------------------------------------------------------------------------------
module pkt_generator
  import apb_pkt_pkg::*;
(
  input  logic        clk,
  input  logic        rst_n,
  input  logic        enable,
  input  logic        start_pulse,
  input  logic        stop_pulse,
  input  logic        continuous,
  input  logic [1:0]  cfg_dst,
  input  logic [1:0]  cfg_src,
  input  logic [7:0]  cfg_len,
  input  logic [7:0]  cfg_seed,
  input  logic [15:0] cfg_interval,
  input  logic [15:0] cfg_count,
  input  logic        out_ready,
  output logic        out_valid,
  output logic        out_sop,
  output logic        out_eop,
  output logic [1:0]  out_dst,
  output logic [1:0]  out_src,
  output logic [7:0]  out_len,
  output logic [31:0] out_data,
  output logic        running,
  output logic [15:0] remaining,
  output logic [1:0]  fsm,
  output logic        done_pulse,
  output logic        start_ack
);

  logic [1:0]  fsm_q;
  logic [7:0]  beat_q;
  logic [15:0] gap_q;
  logic [15:0] left_q;
  logic [15:0] pkt_idx_q;

  logic [7:0] len_n;
  assign len_n = (cfg_len == 8'd0) ? 8'd1 :
                 (cfg_len > LEN_MAX[7:0]) ? LEN_MAX[7:0] : cfg_len;

  logic [15:0] n_pkts;
  assign n_pkts = (cfg_count == 16'd0) ? 16'd1 : cfg_count;

  logic last_beat;
  assign last_beat = (beat_q + 8'd1) >= len_n;

  assign fsm       = fsm_q;
  assign remaining = left_q;
  assign running   = (fsm_q == GEN_SEND) || (fsm_q == GEN_GAP);
  assign out_dst   = cfg_dst;
  assign out_src   = cfg_src;
  assign out_len   = len_n;

  assign out_valid = (fsm_q == GEN_SEND) && enable;
  assign out_sop   = out_valid && (beat_q == 8'd0);
  assign out_eop   = out_valid && last_beat;
  assign out_data  = {cfg_seed, pkt_idx_q[7:0], beat_q, 8'hA5};

  logic fire_start;
```

```
  assign fire_start = start_pulse && enable &&
                      (fsm_q == GEN_IDLE || fsm_q == GEN_DONE);

  assign start_ack  = fire_start;
  assign done_pulse = (fsm_q == GEN_SEND) && enable && last_beat &&
                      (out_ready || 1'b1) && // always accepted by switch
                      !continuous && (left_q == 16'd1);

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      fsm_q     <= GEN_IDLE;
      beat_q    <= 8'd0;
      gap_q     <= 16'd0;
      left_q    <= 16'd0;
      pkt_idx_q <= 16'd0;
    end else if (!enable || stop_pulse) begin
      fsm_q     <= GEN_IDLE;
      beat_q    <= 8'd0;
      gap_q     <= 16'd0;
      left_q    <= 16'd0;
    end else begin
      unique case (fsm_q)
        GEN_IDLE: begin
          beat_q <= 8'd0;
          gap_q  <= 16'd0;
          if (fire_start) begin
            fsm_q     <= GEN_SEND;
            left_q    <= n_pkts;
            pkt_idx_q <= 16'd0;
          end
        end
        GEN_SEND: begin
          if (out_ready || 1'b1) begin
            if (last_beat) begin
              pkt_idx_q <= pkt_idx_q + 16'd1;
              if (!continuous && left_q == 16'd1) begin
                left_q <= 16'd0;
                fsm_q  <= GEN_DONE;
              end else begin
                if (!continuous)
                  left_q <= left_q - 16'd1;
                beat_q <= 8'd0;
                if (cfg_interval == 16'd0)
                  fsm_q <= GEN_SEND;
                else begin
                  gap_q <= 16'd1;
                  fsm_q <= GEN_GAP;
                end
              end
            end else begin
              beat_q <= beat_q + 8'd1;
            end
          end
        end
        GEN_GAP: begin
          if (gap_q >= cfg_interval) begin
            gap_q  <= 16'd0;
            beat_q <= 8'd0;
            fsm_q  <= GEN_SEND;
          end else begin
            gap_q <= gap_q + 16'd1;
          end
        end
        GEN_DONE: begin
          if (fire_start) begin
            fsm_q     <= GEN_SEND;
            left_q    <= n_pkts;
            pkt_idx_q <= 16'd0;
            beat_q    <= 8'd0;
          end
        end
        default: fsm_q <= GEN_IDLE;
      endcase
    end
  end

endmodule
```

rtl/pkt_switch.sv — Four-port switch fabric

```
// ----------------------------------------------------------------------------
// pkt_switch.sv
// Four-port packet switch: dest-port lookup, per-port queues, RX/TX/drop stats.
// ----------------------------------------------------------------------------
module pkt_switch
  import apb_pkt_pkg::*;
(
  input  logic             clk,
  input  logic             rst_n,
  input  logic             switch_en,
  input  logic [N_PORTS-1:0] port_en,
  input  logic [N_PORTS-1:0] fwd_en,
  input  logic             stat_clr,
  input  logic             in_valid,
  input  logic             in_sop,
  input  logic             in_eop,
  input  logic [1:0]       in_dst,
  input  logic [1:0]       in_src,
  input  logic [7:0]       in_len,
  input  logic [31:0]      in_data,
  output logic             in_ready,
  output logic [7:0]       q_level [N_PORTS],
  output logic [15:0]      stat_rx,
  output logic [15:0]      stat_tx,
  output logic [15:0]      stat_drop,
  output logic             ovf_evt,
  output logic             dest_err_evt,
  output logic             drop_evt,
  output logic             fwd_eop_evt
);

  logic dropping_q;
  logic [7:0] q_q [N_PORTS];
  logic [15:0] rx_q, tx_q, drop_q;

  assign in_ready = 1'b1;
  assign stat_rx   = rx_q;
  assign stat_tx   = tx_q;
  assign stat_drop = drop_q;

  genvar gi;
  generate
    for (gi = 0; gi < N_PORTS; gi++) begin : g_ql
      assign q_level[gi] = q_q[gi];
    end
  endgenerate

  logic dest_ok;
  logic queue_ok;
  logic accept;
  logic [1:0] d;
  assign d = in_dst;
  assign dest_ok  = port_en[d] && fwd_en[d] && switch_en;
  assign queue_ok = (q_q[d] < Q_DEPTH[7:0]);
  assign accept   = in_valid && dest_ok && queue_ok && !dropping_q;

  always_comb begin
    ovf_evt      = 1'b0;
    dest_err_evt = 1'b0;
    drop_evt     = 1'b0;
    fwd_eop_evt  = 1'b0;
    if (in_valid && in_sop) begin
      if (!dest_ok)
        dest_err_evt = 1'b1;
      else if (!queue_ok)
        ovf_evt = 1'b1;
    end
    if (in_valid && in_eop) begin
      if (accept)
        fwd_eop_evt = 1'b1;
      else
        drop_evt = 1'b1;
    end
  end

  integer i;
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      dropping_q <= 1'b0;
      rx_q       <= 16'd0;
      tx_q       <= 16'd0;
      drop_q     <= 16'd0;
      for (i = 0; i < N_PORTS; i++)
```

```
        q_q[i] <= 8'd0;
    end else if (stat_clr) begin
      dropping_q <= 1'b0;
      rx_q       <= 16'd0;
      tx_q       <= 16'd0;
      drop_q     <= 16'd0;
      for (i = 0; i < N_PORTS; i++)
        q_q[i] <= 8'd0;
    end else begin
      if (in_valid && in_sop)
        dropping_q <= !(dest_ok && queue_ok);
      else if (in_valid && in_eop)
        dropping_q <= 1'b0;

      if (accept) begin
        if (in_eop) begin
          if (rx_q != 16'hFFFF) rx_q <= rx_q + 16'd1;
          if (tx_q != 16'hFFFF) tx_q <= tx_q + 16'd1;
        end
      end else if (in_valid && in_eop) begin
        if (drop_q != 16'hFFFF) drop_q <= drop_q + 16'd1;
      end

      for (i = 0; i < N_PORTS; i++) begin
        if (accept && (d == i[1:0])) begin
          if (q_q[i] < Q_DEPTH[7:0])
            q_q[i] <= q_q[i] + 8'd1;
        end else if (q_q[i] > 8'd0) begin
          q_q[i] <= q_q[i] - 8'd1;
        end
      end
    end
  end

  logic _unused_src;
  logic [7:0] _unused_len;
  logic [31:0] _unused_data;
  assign _unused_src  = in_src[0];
  assign _unused_len  = in_len;
  assign _unused_data = in_data;

endmodule
```

rtl/apb_pkt_switch.sv — AMBA APB3 CSR top-level

```
// ------------------------------------------------------------------------------
// apb_pkt_switch.sv
// AMBA APB3 CSR front-end for a 4-port packet switch + packet generator.
// ------------------------------------------------------------------------------
module apb_pkt_switch
  import apb_pkt_pkg::*;
(
  input  logic              pclk,
  input  logic              presetn,
  input  logic              psel,
  input  logic              penable,
  input  logic              pwrite,
  input  logic [ADDR_W-1:0] paddr,
  input  logic [DATA_W-1:0] pwdata,
  output logic [DATA_W-1:0] prdata,
  output logic              pready,
  output logic              pslverr,
  output logic              irq
);

  assign pready = 1'b1;

  logic acc, wr, rd, unaligned, unmapped;
  logic [15:0] word_addr;
  assign acc       = psel & penable & pready;
  assign wr        = acc &  pwrite;
  assign rd        = acc & ~pwrite;
  assign unaligned = |paddr[1:0];
  assign unmapped  = paddr > ADDR_MAX;
  assign word_addr = {paddr[15:2], 2'b00};

  logic wr_ok, rd_ok;
  assign wr_ok = wr & ~unaligned & ~unmapped;
  assign rd_ok = rd & ~unaligned & ~unmapped;
  assign pslverr = acc & (unaligned | unmapped);
```

```
logic locked;
logic [31:0] switch_ctrl_q;
logic [3:0]  port_enable_q;
logic [3:0]  fwd_enable_q;
logic [31:0] gen_ctrl_q;
logic [31:0] gen_pkt_cfg_q;
logic [15:0] gen_interval_q;
logic [15:0] gen_count_q;
logic [7:0]  irq_status_q;
logic [7:0]  irq_enable_q;
logic [7:0]  err_status_q;
logic [7:0]  err_enable_q;
logic [15:0] err_count_q;
logic [31:0] err_info_q;

logic wr_switch_ctrl, wr_port_en, wr_fwd_en, wr_gen_ctrl;
logic wr_gen_pkt, wr_gen_int, wr_gen_cnt;
logic wr_irq_status, wr_irq_enable, wr_irq_force;
logic wr_err_status, wr_err_enable, wr_err_inject;
logic wr_lock, wr_soft_reset;

always_comb begin
  wr_switch_ctrl = 1'b0; wr_port_en = 1'b0; wr_fwd_en = 1'b0;
  wr_gen_ctrl = 1'b0; wr_gen_pkt = 1'b0; wr_gen_int = 1'b0; wr_gen_cnt = 1'b0;
  wr_irq_status = 1'b0; wr_irq_enable = 1'b0; wr_irq_force = 1'b0;
  wr_err_status = 1'b0; wr_err_enable = 1'b0; wr_err_inject = 1'b0;
  wr_lock = 1'b0; wr_soft_reset = 1'b0;
  if (wr_ok) begin
    unique case (word_addr)
      ADDR_SWITCH_CTRL: wr_switch_ctrl = 1'b1;
      ADDR_PORT_ENABLE: wr_port_en     = 1'b1;
      ADDR_FWD_ENABLE:  wr_fwd_en      = 1'b1;
      ADDR_GEN_CTRL:    wr_gen_ctrl    = 1'b1;
      ADDR_GEN_PKT_CFG: wr_gen_pkt     = 1'b1;
      ADDR_GEN_INTERVAL:wr_gen_int     = 1'b1;
      ADDR_GEN_COUNT:   wr_gen_cnt     = 1'b1;
      ADDR_IRQ_STATUS:  wr_irq_status  = 1'b1;
      ADDR_IRQ_ENABLE:  wr_irq_enable  = 1'b1;
      ADDR_IRQ_FORCE:   wr_irq_force   = 1'b1;
      ADDR_ERR_STATUS:  wr_err_status  = 1'b1;
      ADDR_ERR_ENABLE:  wr_err_enable  = 1'b1;
      ADDR_ERR_INJECT:  wr_err_inject  = 1'b1;
      ADDR_LOCK:        wr_lock        = 1'b1;
      ADDR_SOFT_RESET:  wr_soft_reset  = 1'b1;
      default: ;
    endcase
  end
end

logic cfg_wr_ok;
assign cfg_wr_ok = wr_ok & ~locked;

logic srst_pulse, stat_clr_pulse, start_pulse, stop_pulse;
assign srst_pulse     = wr_soft_reset & pwdata[0];
assign stat_clr_pulse = wr_switch_ctrl & cfg_wr_ok & pwdata[1];
assign start_pulse    = wr_gen_ctrl & cfg_wr_ok & pwdata[1];
assign stop_pulse     = wr_gen_ctrl & cfg_wr_ok & pwdata[2];

logic [7:0] irq_force_w, err_inject_w;
logic err_clr_count;
assign irq_force_w   = wr_irq_force ? pwdata[7:0] : 8'h00;
assign err_inject_w  = wr_err_inject ? pwdata[7:0] : 8'h00;
assign err_clr_count = wr_err_inject & pwdata[31];

logic switch_en, irq_gen_en, gen_en, gen_cont;
assign switch_en  = switch_ctrl_q[0];
assign irq_gen_en = switch_ctrl_q[3];
assign gen_en     = gen_ctrl_q[0];
assign gen_cont   = gen_ctrl_q[3];

logic        gen_valid, gen_sop, gen_eop, gen_ready;
logic [1:0]  gen_dst, gen_src, gen_fsm;
logic [7:0]  gen_len;
logic [31:0] gen_data;
logic        gen_running, gen_done_pulse, gen_start_ack;
logic [15:0] gen_left;

logic gen_en_eff;
assign gen_en_eff = (cfg_wr_ok && wr_gen_ctrl) ? pwdata[0] : gen_en;

pkt_generator u_gen (
  .clk         (pclk),
```

```
    .rst_n       (presetn),
    .enable      (gen_en_eff),
    .start_pulse (start_pulse),
    .stop_pulse  (stop_pulse | srst_pulse),
    .continuous  (gen_cont),
    .cfg_dst     (gen_pkt_cfg_q[1:0]),
    .cfg_src     (gen_pkt_cfg_q[3:2]),
    .cfg_len     (gen_pkt_cfg_q[15:8]),
    .cfg_seed    (gen_pkt_cfg_q[23:16]),
    .cfg_interval(gen_interval_q),
    .cfg_count   (gen_count_q),
    .out_ready   (gen_ready),
    .out_valid   (gen_valid),
    .out_sop     (gen_sop),
    .out_eop     (gen_eop),
    .out_dst     (gen_dst),
    .out_src     (gen_src),
    .out_len     (gen_len),
    .out_data    (gen_data),
    .running     (gen_running),
    .remaining   (gen_left),
    .fsm         (gen_fsm),
    .done_pulse  (gen_done_pulse),
    .start_ack   (gen_start_ack)
  );

  logic [7:0]  q_level [N_PORTS];
  logic [15:0] stat_rx, stat_tx, stat_drop;
  logic        ovf_evt, dest_err_evt, drop_evt, fwd_eop_evt;

  pkt_switch u_sw (
    .clk          (pclk),
    .rst_n        (presetn),
    .switch_en    (switch_en),
    .port_en      (port_enable_q),
    .fwd_en       (fwd_enable_q),
    .stat_clr     (stat_clr_pulse | srst_pulse),
    .in_valid     (gen_valid),
    .in_sop       (gen_sop),
    .in_eop       (gen_eop),
    .in_dst       (gen_dst),
    .in_src       (gen_src),
    .in_len       (gen_len),
    .in_data      (gen_data),
    .in_ready     (gen_ready),
    .q_level      (q_level),
    .stat_rx      (stat_rx),
    .stat_tx      (stat_tx),
    .stat_drop    (stat_drop),
    .ovf_evt      (ovf_evt),
    .dest_err_evt (dest_err_evt),
    .drop_evt     (drop_evt),
    .fwd_eop_evt  (fwd_eop_evt)
  );

  logic busy;
  assign busy = gen_running || (q_level[0] != 8'd0) || (q_level[1] != 8'd0) ||
                (q_level[2] != 8'd0) || (q_level[3] != 8'd0);

  logic [31:0] switch_status_r;
  always_comb begin
    switch_status_r = 32'h0;
    switch_status_r[0]    = ~gen_running;
    switch_status_r[1]    = busy;
    switch_status_r[2]    = switch_en;
    switch_status_r[3]    = locked;
    switch_status_r[7:4]  = port_enable_q;
    switch_status_r[8]    = gen_running;
    switch_status_r[9]    = |err_status_q;
    switch_status_r[17:16]= gen_fsm;
  end

  logic [31:0] port_status_r;
  always_comb begin
    port_status_r = 32'h0;
    port_status_r[7:0]   = q_level[0];
    port_status_r[15:8]  = q_level[1];
    port_status_r[23:16] = q_level[2];
    port_status_r[31:24] = q_level[3];
  end

  logic [31:0] gen_status_r;
  always_comb begin
```

```
    gen_status_r = 32'h0;
    gen_status_r[0]     = gen_running;
    gen_status_r[1]     = (gen_fsm == GEN_DONE);
    gen_status_r[3:2]   = gen_fsm;
    gen_status_r[23:8]  = gen_left;
  end

  logic [7:0] irq_w1c, err_w1c, err_set, irq_set;
  assign irq_w1c = wr_irq_status ? pwdata[7:0] : 8'h00;
  assign err_w1c = wr_err_status ? pwdata[7:0] : 8'h00;

  always_comb begin
    err_set = 8'h00;
    if (dest_err_evt) err_set[1] = 1'b1;
    if (ovf_evt)      err_set[2] = 1'b1;
    if (|err_inject_w[6:0]) begin
      err_set = err_set | err_inject_w;
      err_set[7] = 1'b1;
    end else if (err_inject_w[7]) begin
      err_set[7] = 1'b1;
    end
  end

  logic [7:0] err_next;
  assign err_next = (err_status_q | err_set) & ~err_w1c;
  logic err_promote;
  assign err_promote = |((err_status_q | err_set) & err_enable_q);

  always_comb begin
    irq_set = irq_force_w;
    if (gen_done_pulse) irq_set[0] = 1'b1;
    if (err_promote)    irq_set[1] = 1'b1;
    if (ovf_evt)        irq_set[2] = 1'b1;
    if (drop_evt)       irq_set[3] = 1'b1;
    if (fwd_eop_evt)    irq_set[4] = 1'b1;
    if (gen_start_ack)  irq_set[6] = 1'b1;
    if ((q_level[0] >= 8'd8) || (q_level[1] >= 8'd8) ||
        (q_level[2] >= 8'd8) || (q_level[3] >= 8'd8))
      irq_set[5] = 1'b1;
  end

  logic [7:0] irq_next;
  assign irq_next = (irq_status_q | irq_set) & ~irq_w1c;

  int unsigned inj_i, code_i;
  logic [15:0] inject_pop;
  logic [7:0]  last_code;
  always_comb begin
    inject_pop = 16'd0;
    for (inj_i = 0; inj_i < 7; inj_i++)
      inject_pop = inject_pop + {15'd0, err_inject_w[inj_i]};
  end
  always_comb begin
    last_code = 8'hFF;
    for (code_i = 0; code_i < 8; code_i++)
      if (err_set[code_i] && last_code == 8'hFF)
        last_code = code_i[7:0];
  end

  always_ff @(posedge pclk or negedge presetn) begin
    if (!presetn) begin
      locked         <= 1'b1;
      switch_ctrl_q  <= 32'h0;
      port_enable_q  <= RST_PORT_ENABLE[3:0];
      fwd_enable_q   <= RST_FWD_ENABLE[3:0];
      gen_ctrl_q     <= 32'h0;
      gen_pkt_cfg_q  <= RST_GEN_PKT_CFG;
      gen_interval_q <= RST_GEN_INTERVAL[15:0];
      gen_count_q    <= RST_GEN_COUNT[15:0];
      irq_status_q   <= 8'h0;
      irq_enable_q   <= 8'h0;
      err_status_q   <= 8'h0;
      err_enable_q   <= 8'h0;
      err_count_q    <= 16'h0;
      err_info_q     <= 32'h0;
    end else if (srst_pulse) begin
      locked         <= 1'b1;
      switch_ctrl_q  <= 32'h0;
      port_enable_q  <= RST_PORT_ENABLE[3:0];
      fwd_enable_q   <= RST_FWD_ENABLE[3:0];
      gen_ctrl_q     <= 32'h0;
      gen_pkt_cfg_q  <= RST_GEN_PKT_CFG;
      gen_interval_q <= RST_GEN_INTERVAL[15:0];
```

```
      gen_count_q    <= RST_GEN_COUNT[15:0];
      irq_status_q   <= 8'h0;
      irq_enable_q   <= 8'h0;
      err_status_q   <= 8'h0;
      err_enable_q   <= 8'h0;
      err_count_q    <= 16'h0;
      err_info_q     <= 32'h0;
    end else begin
      if (wr_lock)
        locked <= (pwdata != UNLOCK_KEY);

      if (cfg_wr_ok) begin
        if (wr_switch_ctrl) begin
          switch_ctrl_q <= pwdata & MASK_SWITCH_CTRL;
        end
        if (wr_port_en)
          port_enable_q <= pwdata[3:0];
        if (wr_fwd_en)
          fwd_enable_q <= pwdata[3:0];
        if (wr_gen_ctrl) begin
          gen_ctrl_q <= pwdata & MASK_GEN_CTRL;
        end
        if (wr_gen_pkt) begin
          gen_pkt_cfg_q <= pwdata & MASK_GEN_PKT_CFG;
          if (pwdata[15:8] == 8'd0)
            gen_pkt_cfg_q[15:8] <= 8'd1;
        end
        if (wr_gen_int)
          gen_interval_q <= pwdata[15:0];
        if (wr_gen_cnt)
          gen_count_q <= pwdata[15:0];
        if (wr_irq_enable)
          irq_enable_q <= pwdata[7:0];
        if (wr_err_enable)
          err_enable_q <= pwdata[7:0];
      end

      irq_status_q <= irq_next;
      err_status_q <= err_next;

      if (err_clr_count) begin
        err_count_q <= 16'h0;
        err_info_q  <= 32'h0;
      end else if (|err_set) begin
        if (|err_inject_w[6:0]) begin
          if (err_count_q > (16'hFFFF - inject_pop))
            err_count_q <= 16'hFFFF;
          else
            err_count_q <= err_count_q + inject_pop;
        end else if (err_count_q != 16'hFFFF) begin
          err_count_q <= err_count_q + 16'd1;
        end
        err_info_q[7:0]   <= last_code;
        err_info_q[15:8]  <= (|err_inject_w) ? 8'd1 : 8'd0;
        err_info_q[17:16] <= gen_dst;
        err_info_q[31:20] <= stat_tx[11:0];
      end
    end
  end

  assign irq = irq_gen_en & |(irq_status_q & irq_enable_q);

  logic [31:0] rdata_mux;
  always_comb begin
    rdata_mux = 32'h0;
    unique case (word_addr)
      ADDR_DEVICE_ID:     rdata_mux = RST_DEVICE_ID;
      ADDR_VERSION:       rdata_mux = RST_VERSION;
      ADDR_SWITCH_STATUS: rdata_mux = switch_status_r;
      ADDR_PORT_STATUS:   rdata_mux = port_status_r;
      ADDR_GEN_STATUS:    rdata_mux = gen_status_r;
      ADDR_STAT_RX:       rdata_mux = {16'h0, stat_rx};
      ADDR_STAT_TX:       rdata_mux = {16'h0, stat_tx};
      ADDR_STAT_DROP:     rdata_mux = {16'h0, stat_drop};
      ADDR_SWITCH_CTRL:   rdata_mux = switch_ctrl_q;
      ADDR_PORT_ENABLE:   rdata_mux = {28'h0, port_enable_q};
      ADDR_FWD_ENABLE:    rdata_mux = {28'h0, fwd_enable_q};
      ADDR_GEN_CTRL:      rdata_mux = gen_ctrl_q;
      ADDR_GEN_PKT_CFG:   rdata_mux = gen_pkt_cfg_q;
      ADDR_GEN_INTERVAL:  rdata_mux = {16'h0, gen_interval_q};
      ADDR_GEN_COUNT:     rdata_mux = {16'h0, gen_count_q};
      ADDR_IRQ_STATUS:    rdata_mux = {24'h0, irq_status_q};
      ADDR_IRQ_ENABLE:    rdata_mux = {24'h0, irq_enable_q};
```

```
      ADDR_IRQ_FORCE:     rdata_mux = 32'h0;
      ADDR_ERR_STATUS:    rdata_mux = {24'h0, err_status_q};
      ADDR_ERR_ENABLE:    rdata_mux = {24'h0, err_enable_q};
      ADDR_ERR_INJECT:    rdata_mux = 32'h0;
      ADDR_ERR_COUNT:     rdata_mux = {16'h0, err_count_q};
      ADDR_ERR_INFO:      rdata_mux = err_info_q;
      ADDR_LOCK:          rdata_mux = {31'h0, locked};
      ADDR_SOFT_RESET:    rdata_mux = 32'h0;
      default:            rdata_mux = 32'h0;
    endcase
  end

  always_comb begin
    if (psel && !pwrite) begin
      if (unaligned || unmapped) prdata = 32'h0;
      else                       prdata = rdata_mux;
    end else begin
      prdata = 32'h0;
    end
  end

endmodule
```